Лабораторная работа №4 Линейная алгебра

Компьютерный практикум по статистическому анализу данных

Беспутин Глеб Антонович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Поэлементные операции 7
3.2 Характеристики матрицы 7
3.3 Нормы, расстояние и угол 7
3.4 Факторизации 8
3.5 Структура и стоимость 8
3.6 Продуктивные матрицы 9

4 Выполнение лабораторной работы **10**
4.1 Повторение примеров из раздела 4.2 10
4.2 Задания для самостоятельной работы (раздел 4.4) 27

5 Результаты **44**
5.1 Примеры раздела 4.2 44
5.2 Произведение векторов 45
5.3 Системы линейных уравнений 45
5.4 Операции с матрицами 47
5.5 Линейные модели экономики 48

6 Выводы **50**

Список литературы **52**

Список иллюстраций

Список таблиц

1 Цель работы

Изучить средства Julia для операций линейной алгебры и сравнить стоимость этих операций для плотных матриц и матриц специальной структуры.

2 Задание

1. Повторить примеры раздела 4.2:

   - поэлементные суммы, произведения и средние;
   - транспонирование, след, ранг, определитель, обратная и псевдообратная матрицы;
   - нормы векторов и матриц, расстояние, угол, повороты;
   - матричное произведение, единичная матрица, скалярное произведение;
   - LU-, QR- и спектральное разложения, специальные структуры и замер времени;
   - системы с рациональными элементами.

2. Выполнить задания раздела 4.4:

   - скалярное и внешнее произведения вектора;
   - системы с двумя и тремя неизвестными;
   - диагональный вид, степени и корни матриц, собственные значения заданной матрицы 5×5;
   - проверка продуктивности матриц линейной модели экономики.

Случайные матрицы порождаются с зерном `Random.seed!(42)`, поэтому прогон повторяется.

3 Теоретическое введение

3.1 Поэлементные операции

`sum` и `prod` сворачивают массив целиком. Аргумент `dims` ограничивает свёртку строками или столбцами. Среднее считает `mean` из пакета `Statistics`: это сумма, делённая на число элементов вдоль выбранного направления.

3.2 Характеристики матрицы

Пакет `LinearAlgebra` даёт транспонирование (`transpose`), след (`tr`), диагональ (`diag`), ранг (`rank`), определитель (`det`) и обратную матрицу (`inv`). У прямоугольной матрицы обратной нет; `pinv` строит псевдообратную Мура–Пенроуза.

3.3 Нормы, расстояние и угол

Евклидова норма вектора — корень из суммы квадратов координат, в Julia это `norm(x)`. p-норма задаётся вторым аргументом. Норма матрицы как оператора считается через `opnorm`. Расстояние между векторами равно норме разности. Косинус угла:

$$\cos\theta = \frac{X^{\top} Y}{\|X\|_2 \, \|Y\|_2}.$$

`rot180` разворачивает матрицу на 180°, `reverse` вдоль `dims=1` или `dims=2` переворачивает строки или столбцы.

3.4 Факторизации

LU-разложение записывает матрицу как $PA = LU$ с перестановкой P, нижней треугольной L и верхней треугольной U. QR-разложение даёт $A = QR$ с ортогональной Q ($Q^\top Q = E$) и верхней треугольной R. Спектральное разложение симметричной матрицы: $A = PDP^\top$, где на диагонали D стоят собственные значения, а столбцы P — ортонормированные собственные векторы.

Оператор \ решает систему $Ax = b$. Тот же оператор применяется к объекту факторизации, без повторного разложения.

3.5 Структура и стоимость

`eigvals` для точно симметричной матрицы вызывает симметричный алгоритм. Сдвиг одного элемента на несколько машинных эпсилон делает матрицу несимметричной, и вызывается более тяжёлый общий метод. Обёртка `Symmetric` снова включает симметричный алгоритм: читается один треугольник.

`SymTridiagonal` хранит только главную диагональ и одну боковую. Матрица порядка 10^6 в таком виде занимает около 16 МБ. Плотная копия того же порядка потребовала бы порядка 8 ТБ и закончилась бы нехваткой памяти.

Тип `Rational{BigInt}` позволяет решать систему точно, без перевода в `Float64` и без переполнения машинных целых.

3.6 Продуктивные матрицы

Линейная модель экономики имеет вид $x - Ax = y$, то есть $(E - A)x = y$. Матрица A продуктивна, если при любой $y \geq 0$ решение x неотрицательно. Для этого достаточно решить систему на базисных правых частях e_i: любая неотрицательная y есть их комбинация с неотрицательными коэффициентами. Если $E - A$ вырождена, решение есть не при всякой правой части, и матрица не продуктивна.

Эквивалентные критерии: все элементы $(E-A)^{-1}$ неотрицательны; модули всех собственных значений A строго меньше единицы.

4 Выполнение лабораторной работы

4.1 Повторение примеров из раздела 4.2

Пакеты `Statistics`, `LinearAlgebra` и `BenchmarkTools` подключены в окружении проекта (`add_packages.jl`). В методичке их добавляют через `Pkg.add` при первом запуске; здесь они уже стоят в `Project.toml`. Случайные матрицы фиксируются зерном, чтобы прогон воспроизводился.

```
using Statistics
using LinearAlgebra
using BenchmarkTools
using Random

Random.seed!(42)

function showval(title, value)
    println(title)
    show(stdout, MIME"text/plain"(), value)
    println("\n")
end
```

```
showval (generic function with 1 method)
```

4.1.1 Поэлементные операции над многомерными массивами (раздел 4.2.1)

Для матрицы 4×3 считаются суммы и произведения всех элементов, а также суммы и произведения по строкам и по столбцам. Средние значения даёт пакет `Statistics`.

```
a = rand(1:20, (4, 3))
showval("Матрица a:", a)
showval("Сумма всех элементов:", sum(a))
showval("Сумма по столбцам:", sum(a, dims=1))
showval("Сумма по строкам:", sum(a, dims=2))
showval("Произведение всех элементов:", prod(a))
showval("Произведение по столбцам:", prod(a, dims=1))
showval("Произведение по строкам:", prod(a, dims=2))
showval("Среднее:", mean(a))
showval("Среднее по столбцам:", mean(a, dims=1))
showval("Среднее по строкам:", mean(a, dims=2))
```

```
Матрица a:
4×3 Matrix{Int64}:
 13  14  10
 10   4   6
 10  13  14
 15  14  13

Сумма всех элементов:
136

Сумма по столбцам:
```

```
1×3 Matrix{Int64}:
 48  45  43

Сумма по строкам:
4×1 Matrix{Int64}:
 37
 20
 37
 42

Произведение всех элементов:
2170284480000

Произведение по столбцам:
1×3 Matrix{Int64}:
 19500  10192  10920

Произведение по строкам:
4×1 Matrix{Int64}:
 1820
  240
 1820
 2730

Среднее:
11.333333333333334

Среднее по столбцам:
1×3 Matrix{Float64}:
```

```
 12.0  11.25  10.75

Среднее по строкам:
4×1 Matrix{Float64}:
 12.333333333333334
  6.666666666666667
 12.333333333333334
 14.0
```

4.1.2 Транспонирование, след, ранг, определитель и инверсия (раздел 4.2.2)

Эти операции выполняет пакет `LinearAlgebra`. Для прямоугольной матрицы обратная не определена, вместо неё берётся псевдообратная `pinv`.

```
b = rand(1:20, (4, 4))
showval("Матрица b:", b)
showval("Транспонирование:", transpose(b))
showval("След:", tr(b))
showval("Диагональ:", diag(b))
showval("Ранг:", rank(b))
showval("Обратная матрица:", inv(b))
showval("Определитель:", det(b))
showval("Псевдообратная к прямоугольной a:", pinv(a))
```

```
Матрица b:
4×4 Matrix{Int64}:
  7   6   2   8
  6   4  11   7
 11   9   4   9
```

```
  2  10  13  18

Транспонирование:
4×4 transpose(::Matrix{Int64}) with eltype Int64:
 7   6  11   2
 6   4   9  10
 2  11   4  13
 8   7   9  18

След:
33

Диагональ:
4-element Vector{Int64}:
  7
  4
  4
 18

Ранг:
4

Обратная матрица:
4×4 Matrix{Float64}:
  0.220806   0.103009   -0.0887302  -0.0938297
 -0.665477  -0.206527    0.514533    0.118817
 -0.18154    0.081591    0.0683325   0.0147884
  0.476288   0.0443651  -0.325344   -0.0107088
```

```
Определитель:
-1960.9999999999998

Псевдообратная к прямоугольной a:
3×4 Matrix{Float64}:
  0.0262499   0.11696    -0.0977858   0.0311341
  0.15734    -0.162652   -0.0439579   0.0013789
 -0.173366    0.0608313   0.176741   -0.00813067
```

4.1.3 Нормы, расстояние, угол, повороты (раздел 4.2.3)

Евклидова норма вектора:

$$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + ... + x_n^2}.$$

p-норма:

$$\|\vec{X}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

Расстояние между векторами — это норма их разности, угол считается через скалярное произведение.

```
X = [2, 4, -5]
showval("Евклидова норма X:", norm(X))
showval("1-норма X:", norm(X, 1))

Y = [1, -1, 3]
showval("Расстояние ||X - Y||:", norm(X - Y))
showval("Проверка через определение:", sqrt(sum((X - Y) .^ 2)))
```

```
cos_theta = (transpose(X) * Y) / (norm(X) * norm(Y))
showval("Косинус угла:", cos_theta)
showval("Угол, рад:", acos(cos_theta))

d = [5 -4 2; -1 2 3; -2 1 0]
showval("Матрица d:", d)
showval("Спектральная норма:", opnorm(d))
showval("1-норма матрицы:", opnorm(d, 1))
showval("Поворот на 180 градусов:", rot180(d))
showval("Переворот строк:", reverse(d, dims=1))
showval("Переворот столбцов:", reverse(d, dims=2))
```

```
Евклидова норма X:
6.708203932499369

1-норма X:
11.0

Расстояние ||X - Y||:
9.486832980505138

Проверка через определение:
9.486832980505138

Косинус угла:
-0.764093177458341

Угол, рад:
2.4404307889469252
```

```
Матрица d:
3×3 Matrix{Int64}:
  5  -4  2
 -1   2  3
 -2   1  0

Спектральная норма:
7.147682841795258

1-норма матрицы:
8.0

Поворот на 180 градусов:
3×3 Matrix{Int64}:
 0   1  -2
 3   2  -1
 2  -4   5

Переворот строк:
3×3 Matrix{Int64}:
 -2   1  0
 -1   2  3
  5  -4  2

Переворот столбцов:
3×3 Matrix{Int64}:
 2  -4   5
 3   2  -1
```

```
 0   1  -2
```

4.1.4 Матричное умножение, единичная матрица, скалярное произведение (раздел 4.2.4)

```
A = rand(1:10, (2, 3))
B = rand(1:10, (3, 4))
showval("A (2×3):", A)
showval("B (3×4):", B)
showval("A * B:", A * B)
showval("Единичная матрица 3×3:", Matrix{Int}(I, 3, 3))

X = [2, 4, -5]
Y = [1, -1, 3]
showval("dot(X, Y):", dot(X, Y))
showval("X'Y:", X'Y)
```

```
A (2×3):
2×3 Matrix{Int64}:
 4  4  8
 6  2  4

B (3×4):
3×4 Matrix{Int64}:
 1  5  6   2
 1  8  5   3
 7  2  9  10

A * B:
```

```
2×4 Matrix{Int64}:
 64  68  116  100
 36  54   82   58

Единичная матрица 3×3:
3×3 Matrix{Int64}:
 1  0  0
 0  1  0
 0  0  1

dot(X, Y):
-17

X'Y:
-17
```

4.1.5 Факторизации и специальные матричные структуры (раздел 4.2.5)

Система $Ax = b$ с известным решением $x = (1, 1, 1)^{\mathsf{T}}$ решается оператором \. LU- и QR-разложения хранятся своими типами, из них достаются множители. Спектральное разложение считается для симметризованной матрицы $A + A^{\mathsf{T}}$.

```
A = rand(3, 3)
x = fill(1.0, 3)
b = A * x
showval("Решение A\\b:", A \ b)

Alu = lu(A)
```

```
showval("LU-разложение:", Alu)
showval("Матрица перестановок P:", Alu.P)
showval("Вектор перестановок p:", Alu.p)
showval("Множитель L:", Alu.L)
showval("Множитель U:", Alu.U)
showval("Решение через LU:", Alu \ b)
println("||(A\\b) - (Alu\\b)|| = ", norm((A \ b) - (Alu \ b)))
println("det(A) = ", det(A), ", det(Alu) = ", det(Alu))

Aqr = qr(A)
showval("QR-разложение:", Aqr)
showval("Множитель Q:", Aqr.Q)
showval("Множитель R:", Aqr.R)
showval("Q'Q (должна быть единичной):", Matrix(Aqr.Q' * Aqr.Q))

Asym = A + A'
AsymEig = eigen(Asym)
showval("Собственные значения:", AsymEig.values)
showval("Собственные векторы:", AsymEig.vectors)
println("||inv(F) * Asym - I|| = ", norm(inv(AsymEig) * Asym - I(3)))
```

```
Решение A\b:
3-element Vector{Float64}:
 1.0
 0.9999999999999998
 1.0000000000000002

LU-разложение:
LU{Float64, Matrix{Float64}, Vector{Int64}}
```

```
L factor:
3×3 Matrix{Float64}:
 1.0        0.0       0.0
 0.53572    1.0       0.0
 0.964456  -0.638389  1.0
U factor:
3×3 Matrix{Float64}:
 0.638659  0.514138   0.94861
 0.0       0.584176   0.152585
 0.0       0.0       -0.698926

Матрица перестановок P:
3×3 Matrix{Float64}:
 0.0  0.0  1.0
 1.0  0.0  0.0
 0.0  1.0  0.0

Вектор перестановок p:
3-element Vector{Int64}:
 3
 1
 2

Множитель L:
3×3 Matrix{Float64}:
 1.0        0.0       0.0
 0.53572    1.0       0.0
 0.964456  -0.638389  1.0
```

```
Множитель U:
3×3 Matrix{Float64}:
 0.638659  0.514138   0.94861
 0.0       0.584176   0.152585
 0.0       0.0       -0.698926

Решение через LU:
3-element Vector{Float64}:
 1.0
 0.9999999999999998
 1.0000000000000002

||(A\b) - (Alu\b)|| = 0.0
det(A) = -0.26076166722565347, det(Alu) = -0.26076166722565347
QR-разложение:
LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3×3 Matrix{Float64}:
 -0.950974  -0.734183  -0.951598
  0.0        0.692354   0.536796
  0.0        0.0        0.396047

Множитель Q:
3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}

Множитель R:
3×3 Matrix{Float64}:
 -0.950974  -0.734183  -0.951598
```

```
  0.0        0.692354   0.536796
  0.0        0.0        0.396047

Q'Q (должна быть единичной):
3×3 Matrix{Float64}:
  1.0          -1.11022e-16   2.22045e-16
 -3.33067e-16   1.0          -2.22045e-16
  0.0          -1.11022e-16   1.0

Собственные значения:
3-element Vector{Float64}:
 -1.0850289097995671
  0.5724199253275271
  3.339978728461213

Собственные векторы:
3×3 Matrix{Float64}:
  0.697559  -0.426123  0.576048
 -0.699439  -0.57948   0.418317
 -0.155554   0.69471   0.702268

||inv(F) * Asym - I|| = 1.039289729849469e-14
```

Дальше та же идея на матрице 1000×1000. Если матрица точно симметрична, `eigvals` идёт по симметричному алгоритму. Сдвиг одного элемента на 5ε симметрию ломает. Обёртка `Symmetric` возвращает быстрый алгоритм, даже если в данных есть шум: читается только один треугольник.

```
n = 1000
A = randn(n, n)
Asym = A + A'
println("issymmetric(Asym) = ", issymmetric(Asym))

Asym_noisy = copy(Asym)
Asym_noisy[1, 2] += 5eps()
println("issymmetric(Asym_noisy) = ", issymmetric(Asym_noisy))

Asym_explicit = Symmetric(Asym_noisy)
println("Собственные значения, симметричная матрица:")
@btime eigvals($Asym) samples=2 evals=1;
println("Собственные значения, зашумлённая матрица:")
@btime eigvals($Asym_noisy) samples=2 evals=1;
println("Собственные значения, явный тип Symmetric:")
@btime eigvals($Asym_explicit) samples=2 evals=1;
```

```
issymmetric(Asym) = true
issymmetric(Asym_noisy) = false
Собственные значения, симметричная матрица:
  33.511 ms (18 allocations: 7.98 MiB)
Собственные значения, зашумлённая матрица:
  269.560 ms (27 allocations: 7.93 MiB)
Собственные значения, явный тип Symmetric:
  30.498 ms (18 allocations: 7.98 MiB)
```

Трёхдиагональная симметричная матрица порядка 10^6 в плотном виде не помещается в память (порядка 8 ТБ). Тип `SymTridiagonal` хранит только диагональ и поддиагональ, и `eigmax` для него считается быстро. Плотную копию

здесь не строим: для такого порядка это `OutOfMemoryError`. Разницу объёма памяти видно на уменьшенном порядке.

```
n = 1_000_000
A = SymTridiagonal(randn(n), randn(n - 1))
println("Память SymTridiagonal 10^6: ", Base.summarysize(A), " байт")
println("Наибольшее собственное значение:")
λmax = @btime eigmax($A) samples=2 evals=1
println("eigmax = ", λmax)

n_demo = 2000
A_demo = SymTridiagonal(randn(n_demo), randn(n_demo - 1))
B_demo = Matrix(A_demo)
println("Память SymTridiagonal n=2000: ", Base.summarysize(A_demo), " байт")
println("Память плотной копии n=2000: ", Base.summarysize(B_demo), " байт")
```

```
Память SymTridiagonal 10^6: 16000088 байт
Наибольшее собственное значение:
  389.401 ms (25 allocations: 183.11 MiB)
eigmax = 6.074779819476323
Память SymTridiagonal n=2000: 32088 байт
Память плотной копии n=2000: 32000048 байт
```

4.1.6 Общая линейная алгебра (раздел 4.2.6)

Рациональные элементы задаются через `//`. Чтобы не словить переполнение на целых фиксированной разрядности, берётся `Rational{BigInt}`. Решение системы не переводится в `Float64`.

```
Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3)) / 10
x = fill(1, 3)
b = Arational * x
showval("Рациональная матрица:", Arational)
showval("Правая часть b:", b)
showval("Решение Arational\\b:", Arational \ b)
showval("LU над рациональными числами:", lu(Arational))
```

```
Рациональная матрица:
3×3 Matrix{Rational{BigInt}}:
 3//5    1     1//5
 3//10  1//10   1
 4//5   3//10  3//5

Правая часть b:
3-element Vector{Rational{BigInt}}:
  9//5
  7//5
 17//10

Решение Arational\b:
3-element Vector{Rational{BigInt}}:
 1
 1
 1

LU над рациональными числами:
LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
```

```
3×3 Matrix{Rational{BigInt}}:
  1      0     0
 3//4    1     0
 3//8  -1//62  1
U factor:
3×3 Matrix{Rational{BigInt}}:
 4//5   3//10    3//5
  0    31//40   -1//4
  0      0     239//310
```

4.2 Задания для самостоятельной работы (раздел 4.4)

4.2.1 4.4.1. Произведение векторов

Скалярное произведение вектора на себя пишется в dot_v. Внешнее произведение — в outer_v. Для столбца v это матрица $vv^{\top}$.

```
v = [2, 4, -5]
dot_v = dot(v, v)
outer_v = v * v'
showval("v:", v)
showval("dot_v:", dot_v)
showval("outer_v:", outer_v)
```

```
v:
3-element Vector{Int64}:
  2
  4
```

```
 -5

dot_v:
45

outer_v:
3×3 Matrix{Int64}:
   4    8  -10
   8   16  -20
 -10  -20   25
```

4.2.2 4.4.2. Системы линейных уравнений

У системы $Ax = b$ три исхода, их различает ранг:

- `rank(A) < rank([A b])` — система несовместна, решений нет;
- `rank(A) = rank([A b]) = n` — одно решение, его даёт `\`;
- `rank(A) = rank([A b]) < n` — бесконечно много решений.

В последнем случае печатается нормальное частное решение $x_0 = A^{+}b$ и базис ядра: общее решение $x_0 + Nc$. Для несовместной системы `\` на квадратной вырожденной матрице бросает `SingularException`, поэтому вместо него считается псевдорешение.

```
function analyze_slau(title, A, b)
    println("=== $title ===")
    Af = float.(Matrix(A))
    bf = float.(vec(b))
    rA = rank(Af)
    rAb = rank(hcat(Af, bf))
```

```
    n = size(Af, 2)
    println("rank(A) = $rA, rank([A b]) = $rAb, неизвестных = $n")
    if rA < rAb
        x = pinv(Af) * bf
        println("Система несовместна, решений нет.")
        println("Псевдорешение МНК: ", x)
        println("Невязка ||Ax - b|| = ", norm(Af * x - bf))
    elseif rA == n
        x = Af \ bf
        println("Единственное решение: ", x)
        println("Проверка Ax = ", Af * x)
    else
        x = pinv(Af) * bf
        N = nullspace(Af)
        println("Бесконечно много решений, свобода = $(n - rA).")
        println("Нормальное частное решение x0 = ", x)
        println("Базис ядра:")
        show(stdout, MIME"text/plain"(), N)
        println()
        println("Проверка Ax0 = ", Af * x)
    end
    println()
end
```

```
analyze_slau (generic function with 1 method)
```

Системы с двумя неизвестными.

```
analyze_slau("2a) x + y = 2, x - y = 3", [1 1; 1 -1], [2, 3])
analyze_slau("2b) x + y = 2, 2x + 2y = 4", [1 1; 2 2], [2, 4])
analyze_slau("2c) x + y = 2, 2x + 2y = 5", [1 1; 2 2], [2, 5])
analyze_slau("2d) три одинаковых уравнения x + y = 1", [1 1; 2 2; 3 3], [1, 2, 3])
analyze_slau("2e) x + y = 2, 2x + y = 1, x - y = 3", [1 1; 2 1; 1 -1], [2, 1, 3])
analyze_slau("2f) x + y = 2, 2x + y = 1, 3x + 2y = 3", [1 1; 2 1; 3 2], [2, 1, 3])
```

```
=== 2a) x + y = 2, x - y = 3 ===
rank(A) = 2, rank([A b]) = 2, неизвестных = 2
Единственное решение: [2.5, -0.5]
Проверка Ax = [2.0, 3.0]

=== 2b) x + y = 2, 2x + 2y = 4 ===
rank(A) = 1, rank([A b]) = 1, неизвестных = 2
Бесконечно много решений, свобода = 1.
Нормальное частное решение x0 = [0.9999999999999996, 0.9999999999999996]
Базис ядра:
2×1 Matrix{Float64}:
 -0.7071067811865475
  0.7071067811865475
Проверка Ax0 = [1.9999999999999991, 3.9999999999999982]

=== 2c) x + y = 2, 2x + 2y = 5 ===
rank(A) = 1, rank([A b]) = 2, неизвестных = 2
Система несовместна, решений нет.
Псевдорешение МНК: [1.1999999999999995, 1.1999999999999995]
Невязка ||Ax - b|| = 0.4472135954999579

=== 2d) три одинаковых уравнения x + y = 1 ===
```

```
rank(A) = 1, rank([A b]) = 1, неизвестных = 2
Бесконечно много решений, свобода = 1.
Нормальное частное решение x0 = [0.4999999999999999, 0.5]
Базис ядра:
2×1 Matrix{Float64}:
 -0.7071067811865477
  0.7071067811865475
Проверка Ax0 = [0.9999999999999999, 1.9999999999999998, 2.9999999999999996]

=== 2e) x + y = 2, 2x + y = 1, x - y = 3 ===
rank(A) = 2, rank([A b]) = 3, неизвестных = 2
Система несовместна, решений нет.
Псевдорешение МНК: [1.5000000000000004, -0.9999999999999998]
Невязка ||Ax - b|| = 1.8708286933869707

=== 2f) x + y = 2, 2x + y = 1, 3x + 2y = 3 ===
rank(A) = 2, rank([A b]) = 2, неизвестных = 2
Единственное решение: [-0.9999999999999989, 2.9999999999999982]
Проверка Ax = [1.9999999999999993, 1.0000000000000004, 3.0]
```

Системы с тремя неизвестными.

```
analyze_slau("3a) два уравнения, три неизвестных", [1 1 1; 1 -1 -2], [2, 3])
analyze_slau("3b) x+y+z=2, 2x+2y-3z=4, 3x+y+z=1", [1 1 1; 2 2 -3; 3 1 1], [2, 4, 1])
analyze_slau("3c) x+y+z=1, x+y+2z=0, 2x+2y+3z=1", [1 1 1; 1 1 2; 2 2 3], [1, 0, 1])
analyze_slau("3d) x+y+z=1, x+y+2z=0, 2x+2y+3z=0", [1 1 1; 1 1 2; 2 2 3], [1, 0, 0])
```

```
=== 3a) два уравнения, три неизвестных ===
rank(A) = 2, rank([A b]) = 2, неизвестных = 3
Бесконечно много решений, свобода = 1.
```

```
Нормальное частное решение x0 = [2.2142857142857144, 0.35714285714285676, -
0.5714285714285716]
Базис ядра:
3×1 Matrix{Float64}:
  0.2672612419124245
 -0.8017837257372732
  0.5345224838248487
Проверка Ax0 = [1.9999999999999996, 3.000000000000001]

=== 3b) x+y+z=2, 2x+2y-3z=4, 3x+y+z=1 ===
rank(A) = 3, rank([A b]) = 3, неизвестных = 3
Единственное решение: [-0.5, 2.5, 0.0]
Проверка Ax = [2.0, 4.0, 1.0]

=== 3c) x+y+z=1, x+y+2z=0, 2x+2y+3z=1 ===
rank(A) = 2, rank([A b]) = 2, неизвестных = 3
Бесконечно много решений, свобода = 1.
Нормальное частное решение x0 = [0.9999999999999994, 1.0000000000000016, -
1.0000000000000007]
Базис ядра:
3×1 Matrix{Float64}:
  0.7071067811865481
 -0.7071067811865469
 -5.551115123125783e-16
Проверка Ax0 = [1.0000000000000002, -4.440892098500626e-16, 1.0]

=== 3d) x+y+z=1, x+y+2z=0, 2x+2y+3z=0 ===
rank(A) = 2, rank([A b]) = 3, неизвестных = 3
Система несовместна, решений нет.
```

```
Псевдорешение МНК: [0.8333333333333326, 0.8333333333333345, -1.0000000000000002]
Невязка ||Ax - b|| = 0.5773502691896256
```

4.2.3 4.4.3. Операции с матрицами

Симметричная матрица приводится к диагональному виду спектральным разложением $A = PDP^{\top}$: на диагонали D стоят собственные значения, столбцы P — ортонормированные собственные векторы.

```
function to_diagonal(title, A)
    println("=== $title ===")
    Af = float.(Matrix(A))
    F = eigen(Symmetric(Af))
    D = Diagonal(F.values)
    P = F.vectors
    println("Собственные значения: ", F.values)
    println("Диагональная матрица D:")
    show(stdout, MIME"text/plain"(), D)
    println()
    println("||P*D*P' - A|| = ", norm(P * Matrix(D) * P' - Af))
    println()
end

to_diagonal("a) [1 -2; -2 1]", [1 -2; -2 1])
to_diagonal("b) [1 -2; -2 3]", [1 -2; -2 3])
to_diagonal("c) матрица 3×3", [1 -2 0; -2 1 2; 0 2 0])
```

```
=== a) [1 -2; -2 1] ===
Собственные значения: [-1.0, 3.0]
Диагональная матрица D:
```

```
2×2 Diagonal{Float64, Vector{Float64}}:
 -1.0   ⋅
   ⋅   3.0
||P*D*P' - A|| = 6.473657049138938e-16

=== b) [1 -2; -2 3] ===
Собственные значения: [-0.2360679774997897, 4.23606797749979]
Диагональная матрица D:
2×2 Diagonal{Float64, Vector{Float64}}:
 -0.236068   ⋅
   ⋅        4.23607
||P*D*P' - A|| = 1.0877919644084146e-15

=== c) матрица 3×3 ===
Собственные значения: [-2.1413361156553594, 0.5151380471280724, 3.6261980685272936]
Диагональная матрица D:
3×3 Diagonal{Float64, Vector{Float64}}:
 -2.14134   ⋅         ⋅
   ⋅       0.515138   ⋅
   ⋅        ⋅        3.6262
||P*D*P' - A|| = 6.863656983246416e-15
```

Степени и корни матриц. Квадратный корень из положительно определённой матрицы считает `sqrt`. Кубический корень строится по вещественным кубическим корням собственных значений: главная ветвь `^(1/3)` для собственного значения -1 уходит в комплексные числа. У матрицы из пункта (d) одно собственное значение отрицательно, вещественного квадратного корня нет.

```
Ma = [1 -2; -2 1]
showval("a) M^10:", Ma^10)

Mb = [5.0 -2.0; -2.0 5.0]
Sb = sqrt(Mb)
showval("b) sqrt(M):", Sb)
println("||sqrt(M)^2 - M|| = ", norm(Sb^2 - Mb))

Mc = [1.0 -2.0; -2.0 1.0]
Fc = eigen(Symmetric(Mc))
Sc = Fc.vectors * Diagonal(cbrt.(Fc.values)) * Fc.vectors'
showval("c) вещественный кубический корень:", Sc)
println("||S^3 - M|| = ", norm(Sc^3 - Mc))
showval("Главная ветвь M^(1/3):", Mc^(1 / 3))

Md = [1.0 2.0; 2.0 3.0]
showval("d) собственные значения:", eigvals(Symmetric(Md)))
showval("d) главный квадратный корень (комплексный):", sqrt(Md))
```

```
a) M^10:
2×2 Matrix{Int64}:
  29525  -29524
 -29524   29525

b) sqrt(M):
2×2 Matrix{Float64}:
  2.1889   -0.45685
 -0.45685   2.1889
```

```
||sqrt(M)^2 - M|| = 1.2947314098277875e-15
c) вещественный кубический корень:
2×2 Matrix{Float64}:
  0.221125  -1.22112
 -1.22112    0.221125

||S^3 - M|| = 2.4525628233156873e-15
Главная ветвь M^(1/3):
2×2 Symmetric{ComplexF64, Matrix{ComplexF64}}:
  0.971125+0.433013im  -0.471125+0.433013im
 -0.471125+0.433013im   0.971125+0.433013im

d) собственные значения:
2-element Vector{Float64}:
 -0.2360679774997897
  4.23606797749979

d) главный квадратный корень (комплексный):
2×2 Matrix{ComplexF64}:
 0.568864+0.351578im  0.920442-0.217287im
 0.920442-0.217287im   1.48931+0.134291im
```

Собственные значения матрицы A из задания, диагональная матрица из них и нижняя треугольная часть A. `LowerTriangular` — обёртка, она не копирует массив, поэтому создаётся заметно быстрее, чем `tril`.

```
A_task = [
    140 97 74 168 131
    97 106 89 131 36
    74 89 152 144 71
```

```
    168 131 144 54 142
    131 36 71 142 36
]
 = eigvals(A_task)
D = Diagonal()
L = LowerTriangular(A_task)
showval("Собственные значения A:", )
showval("Диагональная матрица из собственных значений:", D)
showval("Нижняя треугольная матрица:", L)

println("Время eigvals:")
@btime eigvals($A_task);
println("Время Diagonal(eigvals):")
@btime Diagonal(eigvals($A_task));
println("Время LowerTriangular (обёртка):")
@btime LowerTriangular($A_task);
println("Время tril (копия):")
@btime tril($A_task);
```

```
Собственные значения A:
5-element Vector{Float64}:
 -128.49322764802145
  -55.88778455305688
   42.75216727931894
   87.16111477514521
  542.4677301466143

Диагональная матрица из собственных значений:
5×5 Diagonal{Float64, Vector{Float64}}:
```

```
 -128.493     ⋅        ⋅        ⋅         ⋅
    ⋅     -55.8878    ⋅        ⋅         ⋅
    ⋅        ⋅      42.7522    ⋅         ⋅
    ⋅        ⋅        ⋅      87.1611     ⋅
    ⋅        ⋅        ⋅        ⋅      542.468

Нижняя треугольная матрица:
5×5 LowerTriangular{Int64, Matrix{Int64}}:
 140    ⋅    ⋅    ⋅   ⋅
  97  106    ⋅    ⋅   ⋅
  74   89  152    ⋅   ⋅
 168  131  144   54   ⋅
 131   36   71  142  36

Время eigvals:
  1.588 μs (13 allocations: 2.44 KiB)
Время Diagonal(eigvals):
  1.586 μs (14 allocations: 2.45 KiB)
Время LowerTriangular (обёртка):
  2.387 ns (0 allocations: 0 bytes)
Время tril (копия):
  22.667 ns (2 allocations: 272 bytes)
```

4.2.4 4.4.4. Линейные модели экономики

Модель $x - Ax = y$, или $(E - A)x = y$. Матрица A продуктивна, если для любой $y \geq 0$ решение x неотрицательно. Достаточно проверить базисные правые части e_i: произвольная $y \geq 0$ является их неотрицательной комбинацией. Если $E - A$ вырождена, решение существует не при любой правой части, и

матрица не продуктивна.

Второй критерий: все элементы $(E - A)^{-1}$ неотрицательны. Третий: модули всех собственных значений строго меньше 1.

```
function report_definition(title, A)
    println("=== $title ===")
    A = float.(Matrix(A))
    n = size(A, 1)
    E = Matrix{Float64}(I, n, n)
    M = E - A
    if rank(M) < n
        println("E - A вырождена: система разрешима не при любой y. Не продуктивна.")
        println()
        return
    end
    productive = true
    for i in 1:n
        y = zeros(n)
        y[i] = 1.0
        x = M \ y
        println("y = e$i → x = ", x)
        if any(x .< -1e-8)
            productive = false
        end
    end
    y = ones(n)
    println("y = 1 → x = ", M \ y)
    println(productive ? "По определению: продуктивна." : "По определению: не продукт
    println()
```

```
end

function report_inverse(title, A)
    println("=== $title ===")
    A = float.(Matrix(A))
    n = size(A, 1)
    B = inv(Matrix{Float64}(I, n, n) - A)
    showval("(E - A)^{-1}:", B)
    productive = minimum(B) >= -1e-8
    println(productive ? "По критерию обратной матрицы: продуктивна." : "По критерию
    println()
end

function report_spectral(title, A)
    println("=== $title ===")
    A = float.(Matrix(A))
    ev = eigvals(A)
    rho = maximum(abs.(ev))
    println("Собственные значения: ", ev)
    println("Спектральный радиус: ", rho)
    println(rho < 1 ? "По спектральному критерию: продуктивна." : "По спектральному к
    println()
end
```

```
report_spectral (generic function with 1 method)
```

Проверка по определению.

```
report_definition("1a) [1 2; 3 4]", [1 2; 3 4])
report_definition("1b) (1/2)[1 2; 3 4]", [1 2; 3 4] / 2)
report_definition("1c) (1/10)[1 2; 3 4]", [1 2; 3 4] / 10)
```

```
=== 1a) [1 2; 3 4] ===
y = e1 → x = [0.5, -0.5]
y = e2 → x = [-0.3333333333333333, -0.0]
y = 1 → x = [0.16666666666666666, -0.5]
По определению: не продуктивна.

=== 1b) (1/2)[1 2; 3 4] ===
y = e1 → x = [0.5, -0.75]
y = e2 → x = [-0.5, -0.25]
y = 1 → x = [-0.0, -1.0]
По определению: не продуктивна.

=== 1c) (1/10)[1 2; 3 4] ===
y = e1 → x = [1.25, 0.625]
y = e2 → x = [0.41666666666666663, 1.875]
y = 1 → x = [1.6666666666666665, 2.5]
По определению: продуктивна.
```

Критерий неотрицательности обратной матрицы.

```
report_inverse("2a) [1 2; 3 1]", [1 2; 3 1])
report_inverse("2b) (1/2)[1 2; 3 1]", [1 2; 3 1] / 2)
report_inverse("2c) (1/10)[1 2; 3 1]", [1 2; 3 1] / 10)
```

```
=== 2a) [1 2; 3 1] ===
(E - A)^{-1}:
```

```
2×2 Matrix{Float64}:
 -0.0  -0.333333
 -0.5   0.0

По критерию обратной матрицы: не продуктивна.

=== 2b) (1/2)[1 2; 3 1] ===
(E - A)^{-1}:
2×2 Matrix{Float64}:
 -0.4  -0.8
 -1.2  -0.4

По критерию обратной матрицы: не продуктивна.

=== 2c) (1/10)[1 2; 3 1] ===
(E - A)^{-1}:
2×2 Matrix{Float64}:
 1.2  0.266667
 0.4  1.2

По критерию обратной матрицы: продуктивна.
```

Спектральный критерий.

```
report_spectral("3a) [1 2; 3 1]", [1 2; 3 1])
report_spectral("3b) (1/2)[1 2; 3 1]", [1 2; 3 1] / 2)
report_spectral("3c) (1/10)[1 2; 3 1]", [1 2; 3 1] / 10)
report_spectral("3d) матрица 3×3", [0.1 0.2 0.3; 0.0 0.1 0.2; 0.0 0.1 0.3])
```

```
=== 3a) [1 2; 3 1] ===
```

```
Собственные значения: [-1.4494897427831779, 3.4494897427831783]
Спектральный радиус: 3.4494897427831783
По спектральному критерию: не продуктивна.

=== 3b) (1/2)[1 2; 3 1] ===
Собственные значения: [-0.7247448713915892, 1.724744871391589]
Спектральный радиус: 1.724744871391589
По спектральному критерию: не продуктивна.

=== 3c) (1/10)[1 2; 3 1] ===
Собственные значения: [-0.14494897427831785, 0.34494897427831783]
Спектральный радиус: 0.34494897427831783
По спектральному критерию: продуктивна.

=== 3d) матрица 3×3 ===
Собственные значения: [0.02679491924311228, 0.1, 0.37320508075688774]
Спектральный радиус: 0.37320508075688774
По спектральному критерию: продуктивна.
```

5 Результаты

5.1 Примеры раздела 4.2

Поэлементные операции на матрице 4×3 со случайными целыми от 1 до 20 дали сумму 136 и среднее $11.\overline{3}$. Суммы по столбцам — 48, 45 и 43. Произведение всех элементов — 2 170 284 480 000.

У матрицы 4×4 след равен 33, ранг равен 4, определитель равен -1961. Обратная матрица и псевдообратная к прямоугольной матрице 4×3 посчитаны.

Для $X = (2, 4, -5)$ и $Y = (1, -1, 3)$ евклидова норма X равна $\sqrt{45} \approx 6.708$, 1-норма равна 11, расстояние $\|X - Y\| \approx 9.487$. Скалярное произведение равно -17, угол составляет около 2.440 радиан. Обе формулы расстояния совпали. `dot(X, Y)` и `X'Y` дали одно и то же число.

Система с заранее известным решением $x = (1, 1, 1)^{\mathsf{T}}$ восстановлена и через `A \ b`, и через LU-объект: нормы разности решений и расхождение определителей равны нулю. Произведение $Q^{\mathsf{T}} Q$ совпало с единичной матрицей с точностью до 10^{-16}. Для симметризованной матрицы $\|F^{-1} A - E\| \approx 10^{-14}$.

Рациональная система 3×3 решена точно: ответ $(1, 1, 1)$ в типе `Rational{BigInt}`, без перехода к числам с плавающей точкой.

Замеры на матрице 1000×1000 (два прогона `@btime` после прогрева):

Матрица	Время `eigvals`
$A + A^{\mathsf{T}}$, точно симметричная	33.9 мс

Матрица	Время `eigvals`
та же матрица с шумом 5ε	282 мс
шум, но тип `Symmetric`	32.5 мс

Общий алгоритм медленнее симметричного примерно в 8–9 раз. Явное объявление структуры возвращает быстрый метод, хотя в элементе `(1, 2)` шум остаётся.

`SymTridiagonal` порядка 10^6 занимает 16 000 088 байт. `eigmax` посчитан за 379 мс, значение около 6.075. Плотную матрицу этого порядка скрипт не размещает. На порядке 2000 плотная копия занимает 32 000 048 байт против 32 088 байт у трёхдиагонального хранения — примерно в тысячу раз больше.

5.2 Произведение векторов

Для $v = (2, 4, -5)$ скалярное произведение v на себя равно `dot_v` $= 45$. Внешнее произведение:

$$\text{outer_v} = v v^{\top} = \begin{pmatrix} 4 & 8 & -10 \\ 8 & 16 & -20 \\ -10 & -20 & 25 \end{pmatrix}.$$

5.3 Системы линейных уравнений

Ранг матрицы и ранг расширенной матрицы разделяют три случая: одно решение, бесконечно много решений, решений нет.

Система	Исход	Результат
$x + y = 2,\ x - y = 3$	одно решение	$x = 5/2,\ y = -1/2$
$x + y = 2,\ 2x + 2y = 4$	бесконечно много	$x + y = 2$

Система	Исход	Результат
$x + y = 2,\ 2x + 2y = 5$	нет решений	невязка МНК ≈ 0.447
$x + y = 1$ (три пропорциональные строки)	бесконечно много	$x + y = 1$
$x + y = 2,\ 2x + y = 1,$ $x - y = 3$	нет решений	невязка ≈ 1.871
$x + y = 2,\ 2x + y = 1,$ $3x + 2y = 3$	одно решение	$x = -1,\ y = 3$
$x + y + z = 2,$ $x - y - 2z = 3$	бесконечно много	$(t,\ 7 - 3t,\ 2t - 5)$
$x + y + z = 2,$ $2x + 2y - 3z = 4,$ $3x + y + z = 1$	одно решение	$x = -1/2,\ y = 5/2,\ z = 0$
$x + y + z = 1,$ $x + y + 2z = 0,$ $2x + 2y + 3z = 1$	бесконечно много	$z = -1,\ x + y = 2$
$x + y + z = 1,$ $x + y + 2z = 0,$ $2x + 2y + 3z = 0$	нет решений	невязка ≈ 0.577

Для совместных неопределённых систем скрипт печатает нормальное частное решение и базис ядра. Подстановка показывает, что частное решение удовлетворяет уравнениям. Общие решения в таблице записаны в явном виде; они согласованы с этим базисом.

5.4 Операции с матрицами

Все три матрицы симметричны и приводятся к диагональному виду: $\|PDP^{\mathsf{T}} - A\|$ имеет порядок 10^{-15}.

Матрица	Собственные значения
$\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}$	-1 и 3
$\begin{pmatrix} 1 & -2 \\ -2 & 3 \end{pmatrix}$	$2 - \sqrt{5}$ и $2 + \sqrt{5}$
$\begin{pmatrix} 1 & -2 & 0 \\ -2 & 1 & 2 \\ 0 & 2 & 0 \end{pmatrix}$	$\approx -2.141, 0.515, 3.626$

Степени и корни:

- $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}^{10} = \begin{pmatrix} 29525 & -29524 \\ -29524 & 29525 \end{pmatrix}$;
- квадратный корень из $\begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix}$ равен $\frac{1}{2}\begin{pmatrix} \sqrt{3}+\sqrt{7} & \sqrt{3}-\sqrt{7} \\ \sqrt{3}-\sqrt{7} & \sqrt{3}+\sqrt{7} \end{pmatrix}$, численно $\begin{pmatrix} 2.189 & -0.457 \\ -0.457 & 2.189 \end{pmatrix}$, и его квадрат совпадает с исходной матрицей;
- вещественный кубический корень из $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}$ построен по вещественным корням собственных значений -1 и 3 и в кубе возвращает исходную матрицу. Главная ветвь `^(1/3)` комплексна, потому что главное значение $\sqrt[3]{-1}$ не вещественно;
- у $\begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}$ собственные значения $2 \pm \sqrt{5}$, одно из них отрицательно, поэтому вещественного квадратного корня нет. `sqrt` возвращает комплексную матрицу.

Собственные значения матрицы пятого порядка из задания:

$$-128.493,\ -55.888,\ 42.752,\ 87.161,\ 542.468.$$

Из них собрана диагональная матрица. Нижняя треугольная часть исходной матрицы получена обёрткой `LowerTriangular`. На этой матрице 5×5 времена такие:

Операция	Время
`eigvals`	1.43 мкс
`Diagonal(eigvals)`	1.48 мкс
`LowerTriangular`	2.1 нс, без выделения памяти
`tril`	21.5 нс, копия массива

Почти всё время уходит на спектр. Обёртка треугольной структуры дешевле копирования `tril` примерно в десять раз и не выделяет память. На порядке 5 это наносекунды; выигрыш от структуры на больших матрицах виден в замерах раздела 4.2.

5.5 Линейные модели экономики

Матрица	Критерий	Вердикт
$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	определение	не продуктивна: при $y = e_1$ есть отрицательная компонента
$\frac{1}{2}\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	определение	не продуктивна

Матрица	Критерий	Вердикт
$\frac{1}{10}\begin{pmatrix}1 & 2\\ 3 & 4\end{pmatrix}$	определение	продуктивна, базисные решения неотрицательны
$\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	$(E-A)^{-1}\geq 0$	не продуктивна, в обратной матрице есть отрицательные элементы
$\frac{1}{2}\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	$(E-A)^{-1}\geq 0$	не продуктивна
$\frac{1}{10}\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	$(E-A)^{-1}\geq 0$	продуктивна
$\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	спектр	не продуктивна, $\rho\approx 3.45$
$\frac{1}{2}\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	спектр	не продуктивна, $\rho\approx 1.72$
$\frac{1}{10}\begin{pmatrix}1 & 2\\ 3 & 1\end{pmatrix}$	спектр	продуктивна, $\rho\approx 0.345$
$\begin{pmatrix}0.1 & 0.2 & 0.3\\ 0 & 0.1 & 0.2\\ 0 & 0.1 & 0.3\end{pmatrix}$	спектр	продуктивна, $\rho\approx 0.373$

Деление матрицы на 2 спектральный радиус не загоняет под единицу. Деление на 10 уже даёт продуктивную матрицу. Три формулировки критерия на общих примерах согласованы.

6 Выводы

1. В Julia поэлементные свёртки, средние, нормы, след, ранг, определитель, обращение и псевдообращение считаются функциями базовой библиотеки и пакетов `LinearAlgebra` и `Statistics`.

2. Оператор \ решает совместную систему полного ранга. LU- и QR-объекты хранят множители и решают ту же систему без повторного разложения. Для симметричной матрицы спектральное разложение восстанавливает её с ошибкой порядка машинного эпсилон.

3. Тип матрицы влияет на скорость сильнее, чем мелкий шум в данных. На порядке 1000 симметричный метод быстрее общего примерно в 8–9 раз. Трёхдиагональное хранение позволяет искать крайнее собственное значение матрицы порядка 10^6, которую нельзя разместить как плотный массив.

4. Рациональная арифметика `Rational{BigInt}` решает небольшую систему точно.

5. Классификация СЛАУ по рангам отделяет единственное решение, бесконечное множество решений и несовместную систему. Для вырожденной квадратной матрицы \ неприменим: он бросает `SingularException`, поэтому несовместный и неопределённый случаи обрабатываются через ранг, псевдообратную и ядро.

6. Симметричные матрицы из задания диагонализуемы ортогональной заменой. Вещественный матричный корень существует не всегда: он строится по вещественным корням собственных значений, а главная ветвь дробной степени может оказаться комплексной.

7. Продуктивность матрицы модели $x - Ax = y$ проверена по определению, по знаку элементов $(E - A)^{-1}$ и по спектральному радиусу. Продуктивны только матрицы с достаточно малыми элементами: коэффициент 1/10 и заданная матрица 3×3.

Список литературы

1. Julia 1.5 Documentation. — 2020. — URL: https://docs.julialang.org/en/v1/.
2. LinearAlgebra · The Julia Language. — URL: https://docs.julialang.org/en/v1/stdlib/LinearAlge
3. Klok H., Nazarathy Y. Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. — 2020. — URL: https://statisticswithjulia.org/.
4. Okten G. First Semester in Numerical Analysis with Julia. — Florida State University, 2019. — DOI: 10.33009/jul.
5. Антонюк В. А. Язык Julia как инструмент исследователя. — М. : Физический факультет МГУ им. М. В. Ломоносова, 2019.
6. Шиндин А. В. Язык программирования математических вычислений Julia. Базовое руководство. — Нижний Новгород : Нижегородский госуниверситет, 2016.
7. Кулябов Д. С. и др. Компьютерный практикум на языке Julia. Лабораторная работа № 4. Линейная алгебра.